

Maestro Proof of Concept (PoC)

SecureShift Guard App - End-to-End Testing Investigation

Dhruvi Patel

Table of Contents

1. Objective.....	1
2. Maestro Installation.....	1
3. Initial YAML Flow Creation	2
4. Emulator Testing Process.....	3
5. Testing Outcome	4
6. Issues Identified.....	5
7. Findings and Analysis.....	6
8. Conclusion.....	7

1. Objective

The purpose of this Proof of Concept (PoC) was to experimentally investigate whether Maestro could be used for end-to-end (E2E) mobile testing within the SecureShift Guard App project.

Unlike the previous Jest testing work completed for component and validation testing, this PoC focused specifically on real mobile interaction testing using an Android emulator.

The goal was not to fully implement Maestro across the project, but instead to:

- install and configure Maestro,
- test emulator compatibility,
- create a small automation flow,
- and identify any limitations or setup challenges.

2. Maestro Installation

Maestro was installed locally using Git Bash.

The installation process completed successfully, and the Maestro CLI became accessible through terminal commands.

The following verification command was used:

```
maestro --version
```

This confirmed that:

- Maestro installed correctly,
- the CLI was functional,
- and the environment was capable of running Maestro commands.

Screenshot Evidence

```
patel@vee_pro9 MINGW64 /c/SecureShift1/SecureShift (dhruvi-0406/feature/document-expiry-tracking)
$ curl -fsSL https://get.maestro.mobile.dev | bash
which: no maestro in (/c/Users/patel/bin:/mingw64/bin:/usr/local/bin:/usr/bin:/bin:/mingw64/bin:/usr/bin:/c/Users/patel/bin:/c/Users/patel/AppData/Local/Programs/Microsoft VS Code:/c/Program Files/Common Files/Oracle/Java/javapath:/c/Python311/Scripts:/c/Python311:/c/WINDOWS/system32:/c/WINDOWS/System32/Wbem:/c/WINDOWS/System32/WindowsPowerShell/v1.0:/c/WINDOWS/System32/OpenSSH:/c/Program Files/dotnet:/cmd:/c/Program Files/PuTTY:/c/ProgramData/chocolatey/bin:/c/Program Files/nodejs:/c/Program Files/Docker/Docker/resources/bin:/c/Users/patel/AppData/Local/Android/Sdk/platform-tools:/c/Users/patel/AppData/Local/Android/Sdk/emulator:/c/msys64/home/patel/.splashkit:/c/msys64/home/patel/.splashkit/lib/win64:/c/msys64/home/patel/.splashkit/lib:/c/Windows/System32:/c/Windows:/c/Windows/System32/Wbem:/c/Windows/System32/WindowsPowerShell/v1.0:/c/Users/patel/AppData/Local/Programs/Microsoft VS Code/bin:/c/Users/patel/AppData/Local/GitHubDesktop/bin:/c/ProgramData/patel/GitHubDesktop/bin:/c/Users/patel/AppData/Roaming/npm:/c/Users/patel/AppData/Local/Microsoft/WindowsApps:/usr/bin/vendor_perl:/usr/bin/core_perl)
* Create distribution directories...
* Downloading...
##### 100.0%
* Checking archive integrity...
* Extracting archive...
* Remove previous installation (if any)
* Copying archive contents...
* Cleaning up...

Attempt update of interactive bash profile on regular UNIX...
Adding maestro to your PATH in /c/Users/patel/.bashrc
Adding maestro to your PATH in /c/Users/patel/.zshrc

Installation was successful!
Please open a new terminal OR run the following in the existing one:

    export PATH="$PATH":"$HOME/.maestro/bin"

Then run the following command:

    maestro

Welcome to Maestro!
```

```
patel@vee_pro9 MINGW64 /c/SecureShift1/SecureShift (dhruvi-0406/feature/document-expiry-tracking)
$ maestro --version
Anonymous analytics enabled. To opt out, set MAESTRO_CLI_NO_ANALYTICS environment variable to any value before running Maestro.

?-----?
Try out our new Analyze with Ai feature.

See what's new:
> https://maestro.mobile.dev/cli/test-suites-and-reports#analyze
Analyze command:
$ maestro test flow-file.yaml --analyze | bash

To disable this notification, set MAESTRO_CLI_ANALYSIS_NOTIFICATION_DISABLED environment variable to "true" before running Maestro.
?-----?

2.5.1
```

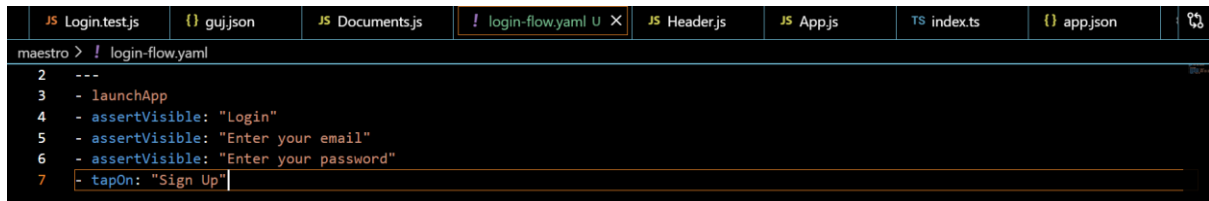
3. Initial YAML Flow Creation

A simple YAML automation flow was created as part of the PoC.

The purpose of this flow was to:

- launch the application,
- verify visible login screen elements,
- and test basic UI interaction behaviour.

The following test flow was used:



```
maestro > ! login-flow.yaml X
2 ---
3 - launchApp
4 - assertVisible: "Login"
5 - assertVisible: "Enter your email"
6 - assertVisible: "Enter your password"
7 - tapOn: "Sign Up"
```

This flow was intentionally kept minimal because the objective was only to test feasibility rather than implement complete application automation.

4. Emulator Testing Process

After setup was completed:

- the Android emulator was launched,
- the SecureShift Guard App was opened through Expo,
- and the Maestro flow was executed using the terminal.

The following command was used:

```
maestro test maestro/login-flow.yaml
```

During execution:

- Maestro successfully detected the Pixel_5 emulator,
- the YAML flow started correctly,
- and the application launch process was initiated.

Screenshot Evidence

```
patel@vee_pro9 MINGW64 /c/SecureShift1/SecureShift (dhruvi-0406/feature/document-expiry-tracking)
$ maestro test maestro/login-flow.yaml
Running on Pixel_5

> Flow: login-flow

+ Launch app "host.exp.exponent"
X Assert that "Login" is visible
  Assert that "Enter your email" is visible
  Assert that "Enter your password" is visible
  Tap on "Sign Up"
```

5. Testing Outcome

The PoC produced a partially successful result.

Several important parts of the setup worked correctly:

- Maestro installed successfully
- Android emulator detection worked
- YAML flow execution started correctly
- Maestro generated logs and debug artifacts
- Automated interaction attempts were made

However, the flow failed during UI assertions.

The main issue identified was that Maestro launched Expo Go (host.exp.exponent) rather than directly opening the SecureShift project instance itself.

Because of this:

- the expected login screen was not visible,
- the "Login" assertion failed,
- and the flow could not continue through the intended login interactions.

Although the automation flow did not fully pass, the PoC still successfully demonstrated the initial feasibility of Maestro within the SecureShift development environment.

Screenshot Evidence

```
patel@vee_pro9 MINGW64 /c/SecureShift1/SecureShift (dhruvi-0406/feature/document-expiry-tracking)
$ maestro test maestro/login-flow.yaml
Running on Pixel_5

> Flow: login-flow
+ Launch app "host.exp.exponent"
X Assert that "Login" is visible
  Assert that "Enter your email" is visible
  Assert that "Enter your password" is visible
  Tap on "Sign Up"

Assertion is false: "Login" is visible

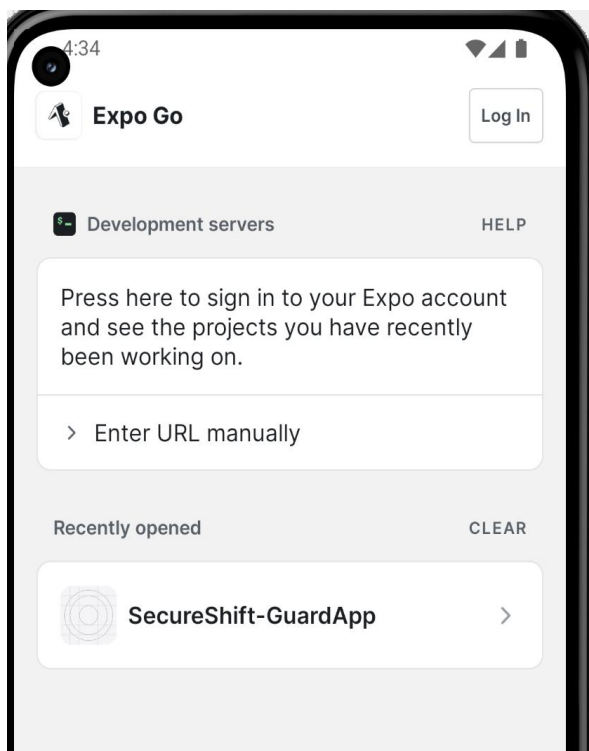
Assertion "'Login' is visible" failed. Check the UI hierarchy in debug artifacts to verify the element state and properties.

Possible causes:
- Element selector may be incorrect - check if there are similar elements with slightly different names/properties.
- Element may be temporarily unavailable due to loading state
- This could be a real regression that needs to be addressed

==== Debug output (logs & screenshots) ====

C:\Users\patel\.maestro\tests\2026-05-17_151559

patel@vee_pro9 MINGW64 /c/SecureShift1/SecureShift (dhruvi-0406/feature/document-expiry-tracking)
```



6. Issues Identified

Several implementation challenges were identified during testing.

Expo Go Launch Behaviour

The primary issue encountered was that Maestro interacted with Expo Go instead of directly launching the running SecureShift project instance.

This caused:

- incorrect launch behaviour,
- missing expected UI elements,
- and failed assertions.

Additional Configuration Requirements

The testing indicated that stable implementation would likely require:

- dedicated development builds,
- improved application launch configuration,
- and potentially stable test IDs for UI elements.

Time and Maintenance Considerations

The PoC also demonstrated that implementing Maestro properly across the entire Guard App would require:

- additional setup time,
- ongoing maintenance,
- and more extensive configuration work.

7. Findings and Analysis

Despite the failed assertions, the PoC successfully confirmed several important findings.

The testing demonstrated that:

- Maestro can successfully run in the SecureShift environment,
- emulator integration functions correctly,
- YAML flows execute correctly,
- and automated interaction attempts can be performed.

The PoC also highlighted important implementation considerations for Expo-based React Native applications.

These findings would be useful for:

- future SecureShift contributors,
- future testing research,

- and possible CI/CD experimentation in later development stages.

8. Conclusion

This Proof of Concept successfully demonstrated the initial feasibility of using Maestro for mobile end-to-end testing research within the SecureShift Guard App.

Although the automation flow did not fully complete successfully, the investigation still achieved its primary purpose by:

- validating Maestro installation,
- testing emulator interaction,
- executing YAML flows,
- identifying limitations,
- and documenting implementation considerations.

The PoC showed that Maestro could potentially become a useful future testing solution for SecureShift, but additional setup, development time, and project-wide planning would be required before full implementation could realistically occur.